AICC efficiency: What still needs to be proven

Measurement research brief · 3 November 2026 · Article status: OPEN DRAFT

Decision: Keep the article open. Round 7 and Round 8/8.1 established feasibility of small capability calls and symbolic toy execution, not net efficiency. We need evidence about three outcomes separately: model tokens, all-in cost, and computing time. No result in this document should be interpreted as a measured AICC advantage.

1. What our research already measured

Research record	Observed result	What it does NOT establish
Round 7: ordinary API vs capability registry	Both passed 14/14; neither needed source inspection	AICC does not yet beat a strong pre-existing library
Round 7: toy symbolic vs Python	Both passed 6/6; examples differed greatly in bytes	No trained native-token model or reliable billed-token savings
Round 8/8.1: composed applications	Both conventional and AICC passed 20/20; mock security 3/3	No multi-run cost/latency comparison or real private isolation
Payload caching illustration	A contract stored once required fewer serialized bytes	Bytes are not inference tokens, billings, or time

The earlier agent results are exploratory; many A/B trials occurred in one model session. In particular, the best existing-library baseline already benefits from concise documentation, tool search, and caching.

2. Independent external evidence: potential, not AICC proof

Study / scope	Reported measurement	Relevance / limitation
Microsoft Research, agent token-cost study (Apr 2026)	Same task could vary by up to 30× in total tokens	Repeated trials required; not an AICC benchmark [1]
EMNLP, enterprise operation resolver (Oct 2026)	54.1% fewer tokens; 43.3% lower median latency versus its baseline	Tool-boundary optimization can matter; not AICC [2]
Microsoft CompactionBench (Oct 2026)	Up to 25.4% lower total cost and 9.6% shorter agent time	Agent-harness efficiency is measurable; not AICC [3]
OpenAI Agents observability documentation	Reports uncached/cached input, output, reasoning, and nested-agent usage	This level of telemetry is the minimum needed for cost claims [4]

Implication: Efficiency optimization is a real research topic with sizable measured gains in other architectures. The effect size for AICC remains unknown.

3. Fair experimental design (factorial)

Hold the model version, reasoning effort, sandbox, test suite, task order distribution, tool limits, and capability implementations constant. Randomize trial order. Make the conventional agent as competent as the AICC agent.

Arm	Composition method	Authoring representation
A - modern baseline	Ordinary packages, documented API, tool search, caching	Python / C# / Java as appropriate

Arm	Composition method	Authoring representation
B - commons only	Same implementations via verified AICC registry and scoped bindings	Same conventional language as A
C - language only	Same tool access as A; no special commons advantage	Proposed symbolic grammar / typed operations
D - combined	AICC registry and reusable bindings	Same symbolic grammar as C
E - future study	Identical task and capability access	Model trained/adapted on dedicated atomic operation tokens

Interpretation: BA isolates the commons effect. CA tests the experimental language surface. DB tests whether symbolic authoring adds value once the commons is available. E cannot be inferred from an ordinary language model generating punctuation.

4. Use tasks that expose real resource-management costs

Pilot: 12 unfamiliar tasks × 3 independent trials per arm. Expanded study: at least 30 tasks × 10 randomized independent trials per arm, with enough repeats to estimate variability. These are proposed study sizes, not published statistical guarantees.

Task family	What must be exercised
Concurrency and queues (Java / C# / Go)	Bounded work, cancellation, ordering, backpressure, timeout and retry boundaries
Memory and buffers (C++ / Rust / C#)	Ownership, object lifetime, copying vs borrowing, peak RSS and cleanup
Data and persistence (Python / Java)	Transactions, isolation, idempotency, exact failure behavior
Images and streams (C++ / Python)	Bounded memory, streaming, decompression limits, data integrity
Mixed public/private composition	Authorized secret capabilities, revocation, provenance, no data leakage

5. Collect raw measurements, not model estimates

Metric class	Required fields / measurement
Tokens	API usage: uncached input, cached input, output, reasoning; root + subagents; all retries
Dollar cost	Actual billed API cost + host/tool charges + amortized registry build/verification cost
Time	Monotonic task wall time; model request duration; discovery/bind/verification; tools and tests
Compute/resources	Sandbox process CPU seconds, peak memory, I/O, API calls and tool invocations
Quality & safety	Hidden-test pass/fail, cross-language conformance, bad reuse, policy failure, retries
Cache/invalidation	Cold vs warm tasks; cache-hit share; revocation and changes to library versions

Record every attempted trial, including errors and incomplete runs. Tokenizer estimates can measure file size but must not be presented as provider-billed inference usage. For agent interfaces lacking usage telemetry, mark monetary inference cost NOT MEASURED.

6. The economics must include the price of the commons

Task cost = uncached-input charge + cached-input charge + output/reasoning charge + tool/CPU charge + verification/retry charge + share of registry creation, curation, and maintenance.

Amortized cost per successful task = total expenditure (including unsuccessful attempts and infrastructure) ÷ number of tasks passing hidden correctness and safety criteria.

An AICC registry can save model-context work but still lose economically if it requires expensive discovery, verification, capability packaging, private execution, or frequent revalidation. Likewise, eliminating already-cached input tokens may save little money at discounted cached-input rates.

7. Decision rule and output

- Primary comparison: paired cost per correctly completed task for B vs A, with confidence intervals and success-rate differences.
- Secondary comparisons: uncached input, cached input, output/reasoning, p50/p95 elapsed task time, CPU seconds, peak memory, and retries.
- Pre-register a meaningful improvement target (illustrative: at least 15% lower all-in cost), require non-inferior correctness and safety, and do not silently revise thresholds after seeing results.
- Report cold-start and warm/repeated workflows separately; show results per task family and per agent, not just an average across providers.
- Do not turn the research article into a final claim until raw telemetry, complete logs, independent reproductions, and failed trials are available.

8. What we can responsibly say in the article now

"Our exploratory AI-agent experiments demonstrated that both conventional reuse and a proposed computational-commons interface can execute the same small tasks correctly. Prior work shows that agent architecture can materially affect token usage, cost and latency. But our experiments have not yet established whether a shared commons or an AI-native language reduces billed tokens, total dollars, or execution time relative to modern tools with effective caching. We therefore treat the proposal as an open research hypothesis."

Sources and verification

[1] Microsoft Research (Apr 2026), How Do AI Agents Spend Your Money?
https://www.microsoft.com/en-us/research/publication/how-do-ai-agents-spend-your-money-analyzing-and-predicting-token-consumption-in-agentic-coding-tasks/

[2] EMNLP Industry Track (Oct 2026), Scaling Enterprise Cloud Agents to Thousands of APIs
https://aclanthology.org/2026.emnlp-industry.179/

[3] Microsoft Research (Oct 2026), CompactionBench
https://www.microsoft.com/en-us/research/publication/benchmarking-long-horizon-agent-context-compaction-with-compactionbench/

[4] OpenAI API, Agents observability and usage
https://developers.openai.com/api/docs/guides/agents-api/observability

[5] OpenAI API, Tool Search and deferred tool loading
https://developers.openai.com/api/docs/guides/tools-tool-search